RAJALAKSHMI
ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME ...SANJAYKUMAR . R.....

ACADEMIC YEAR 2026-27 SEMESTER .....V.....BRANCH....CSD.....

UNIVERSITY REGISTER No. 211624TO1049

Certified that this is the bonafide record of work done by the above student in the Graphics and Multimedia Laboratory during the year 2026 - 2027

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on .....

Internal Examiner **External Examiner**

Ex-No : 01 Date : 21/07/26

BRESENHAM'S LINE DRAWING ALGORITHM FOR |m|<1

Aim:-

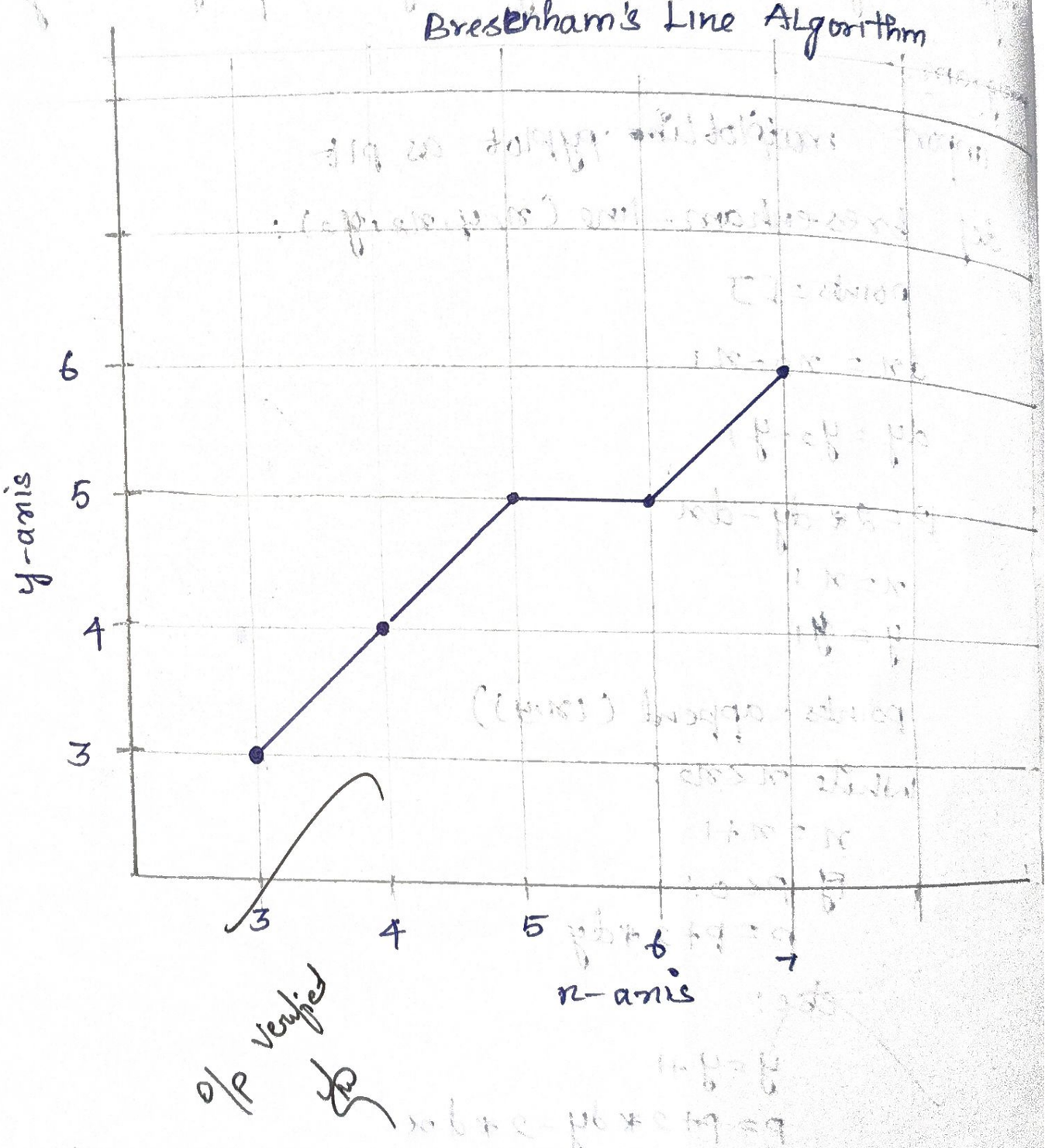
To implement Bresenham's Line drawing algorithm for |m|<1 using python programming.

program:-

```
import matplotlib.pyplot as plt
def bres-enham-line(x1, y1, x2, y2):
    points = []
    dx = x2-x1
    dy = y2-y1
    p = 2*dy-dx
    x = x1
    y = y1
    points.append((x,y))
    while x<x2:
        x = x+1
        if p<0:
            p = p+2*dy
        else:
            y = y+1
            p = p+2*dy-2*dx
        points.append((x,y))
    return points.
x1 = int(input("Enter x1 : "))
y1 = int(input("Enter y1 : "))
x2 = int(input("Enter x2 : "))
```

X: [3, 4, 5, 6, 7]

Y: [3, 4, 5, 5, 6]



o/p verified

```
x_points, y_points = bresenham(x0, y0, xn, yn)
for (x, y in zip(x_points, y_points):
    print(f"({x}, {y})")
plt.figure(figsize=(6,6))
plt.plot(x_points, y_points, marker='8',
linestyle='-', color='blue')
plt.title='-' color='blue')
Algorithm (m<1)")
plt.xlabel("X-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.axis("equal")
plt.show()
```

21/7/20

Result:-

The program was executed successfully

EX-NO-02 Date: 27/01/26

BRESENHAM'S CIRCLE ALGORITHM

Aim:-

To implement bresenham's circle algorithm in python and generate the points of a circle in the given co-ordinates.

PROGRAM:-

```
import matplotlib.pyplot as plt
def bresenham_circle(xc,yc,r):
points = set()
x=0
y = r
d = 3-2*r
def plot_symmetric_points(x_coord, y_coord):
points.add((xc+x_coord, yc+y_coord))
points.add((xc-x_coord, yc+y_coord))
points.add((xc+x_coord, yc-y_coord))
points.add((xc-x_coord, yc-y_coord))
points.add((xc+y_coord, yc+x_coord))
points.add((xc+y_coord, yc-x_coord))
points.add((xc-y_coord, yc-x_coord))
plot_symmetric_points(x,y)
while y>=x:
    x+=1
    if d>0:
    y-=1
    d=d+4*(x-y)+10
    else:
    d=d+4*x+6
    plot_symmetric_points(x,y)
return sorted(list(points))
xc=int(input("Enter the x-coordinate of the centre(xc):"))
yc=int(input("Enter the y-coordinate of the centre(yc):"))
r=int(input("Enter the radius(r):"))
circle_points=bresenham_circle(xc,yc,r)
```

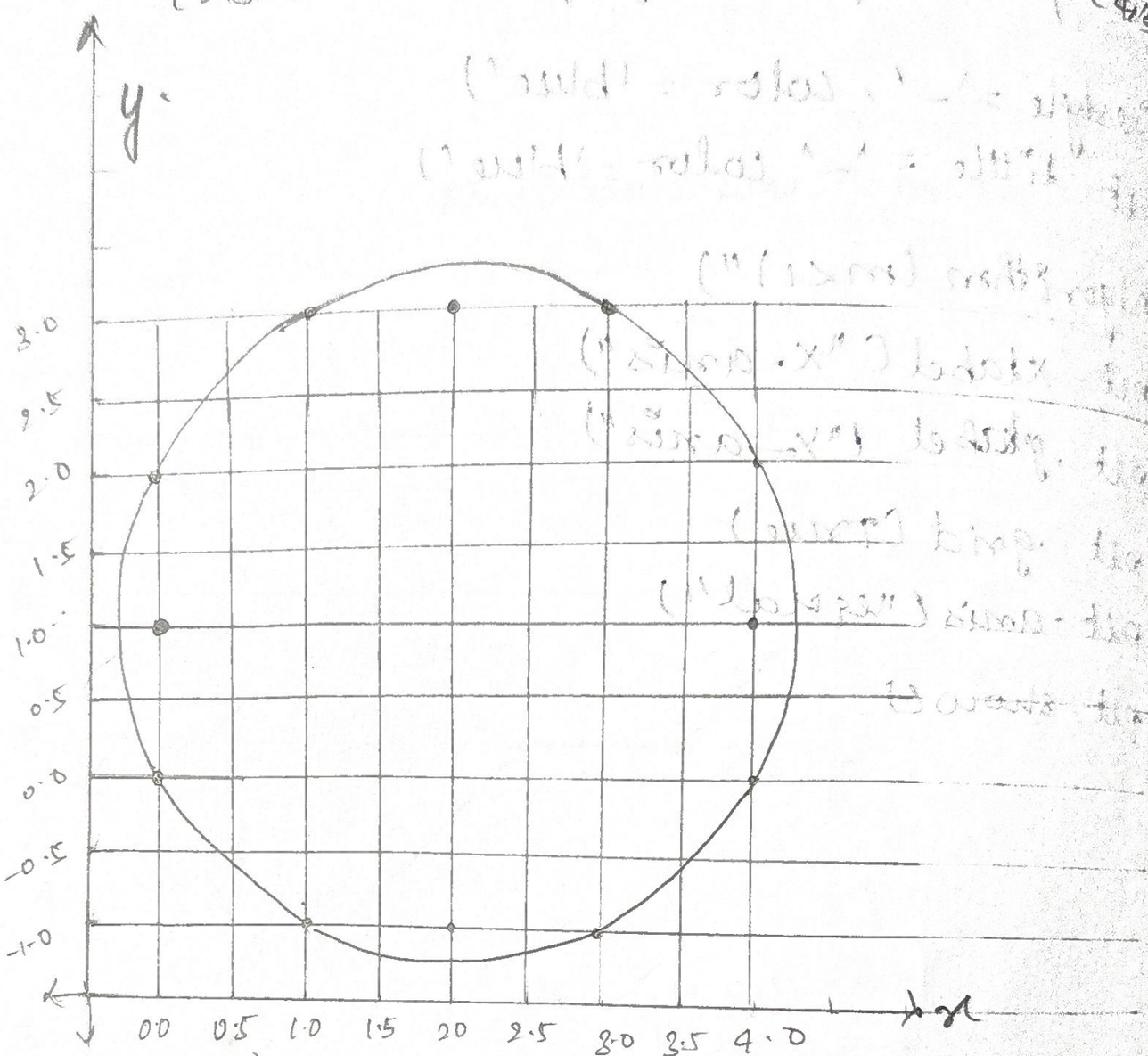
output:

x-co-ord of the centre (xc) : 2

y-co-ord of the centre (yc) : 3

Enter the radius (r) : 2

pt : (0,0) ; (0,1) , (0,2) ; (1,-1) ; (1,3) , (2,-1)
(2,3) , (3,-1) , (3,3) , (4,0) , (4,1) , (4,2)



```
circle_points = bresenham_circle(xc, yc, r)
print("\n points generated for the circle:")
for p in circle_points:
    print(p)
x_coords = [p[0] for p in circle_points]
y_coords = [p[1] for p in circle_points]
plt.figure(figsize=(8,8))
plt.figure(x_coords, y_coords, marker='o', linestyle='')
plt.title("Bresenham's circle drawing Algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
```

4/8/26

Result:-

The Bresenham circle Algorithm was successfully implemented.

EX-NO- 03 Date: 27/07/26

BRESENHAM'S ELLIPSE ALGORITHM

Aim:-

To implement bresenham's ellipse algorithm in python and generate the point of a ellipse in the given co-ordinates.

code:

```
import matplotlib.pyplot as plt
def bresenham_ellipse(xc,yc,rx,ry):
x=0
y=ry
rx2=rx*rx
ry2=ry*ry
p1=ry2-(rx2*ry)+(0.25*rx2)
points=[]
while (2*ry2*x)<(2*rx2*y):
    points.append((x,y))
    if p1<0:
      x=x+1
      p1=p1+(2*ry2*x)+ry2
    else:
      x=x+1
      y=y-1
      p1=p1+(2*ry2*x)-(2*rx2*y)+ry2
p2=(ry2*(x+0.5)*(x+0.5))+\(rx2*(y-1)*
                (y-1))-\(rx2*ry2
while y>=0:
    points.append((x,y))
    if p2>0:
      y=y-1
      p2=p2-(2*rx2*y)+rx2
    else
      x=x+1
      y=y-1
```

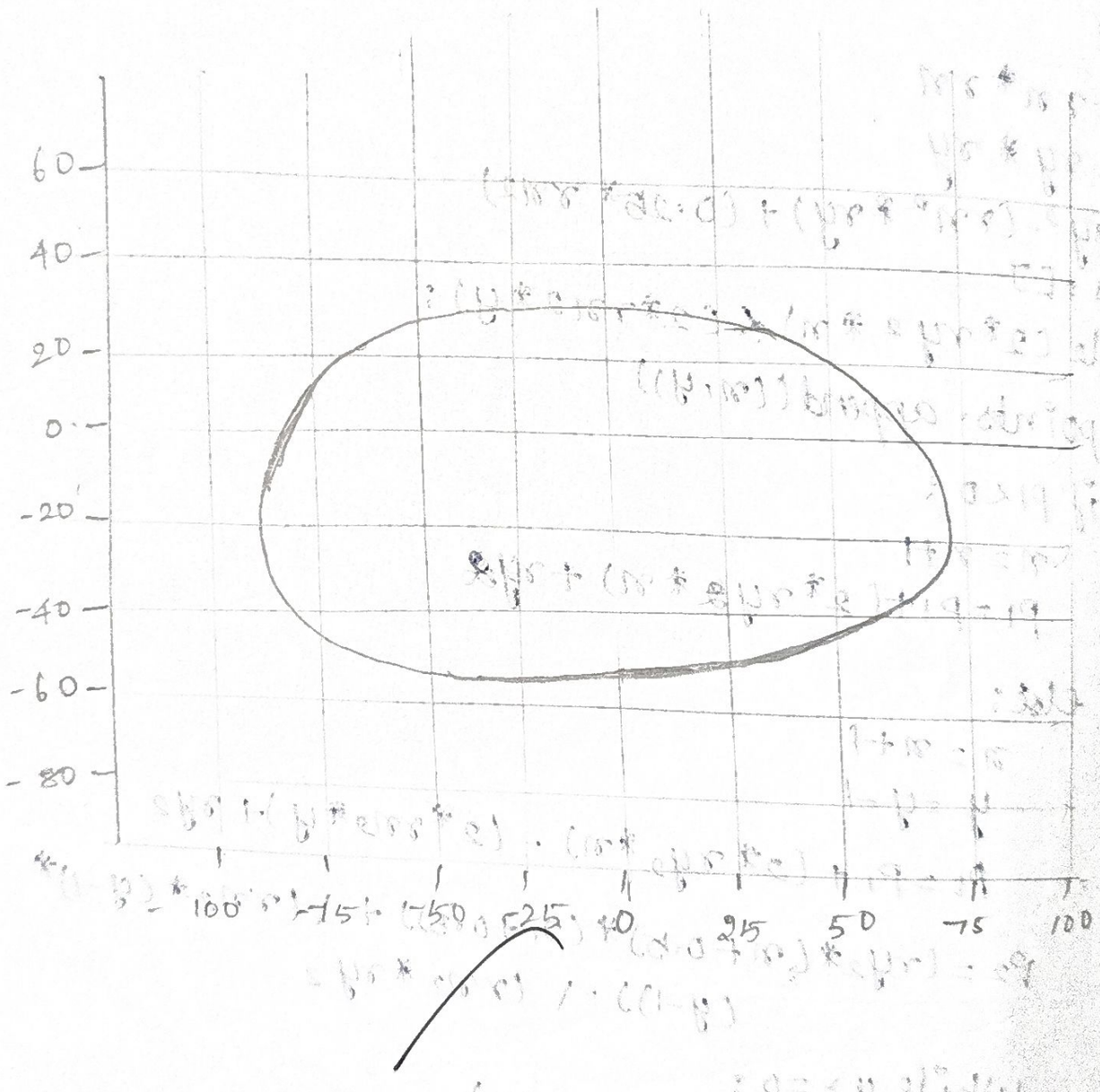
input

Enter x- coordinate of ellipse center $x_c = 50$

Enter y- coordinate of ellipse center $y_c = 50$

Major radius of ellipse $r_x = 40$

Major radius of ellipse $r_y = 20$.



$p_2 = p_2 + (2 * ry_2 * x) - (2 * rx_2 * y) + rx_2$

```
ellipse_points = []
for x, y in points:
    ellipse_points.append((xc + x, yc + y))
    ellipse_points.append((xc - x, yc + y))
    ellipse_points.append((xc + x, yc - y))
    ellipse_points.append((xc - x, yc - y))
return ellipse_points

xc = 0
yc = 0
rx = 100
ry = 50
points = bresenham_ellipse(xc, yc, rx, ry)
x_values = [p[0] for p in points]
y_values = [p[1] for p in points]
plt.scatter(x_values, y_values, s=2)
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.title("Bresenham's Ellipse
Drawing Algorithm")
plt.axis("equal")
plt.grid()
plt.show()
```

Result:

Thus the Bresenham circle Algorithm was Successfully implemented.

4/8/24

EX: 04 Date: 04/08/26.

create a object using Bresenhan's Algorithm.

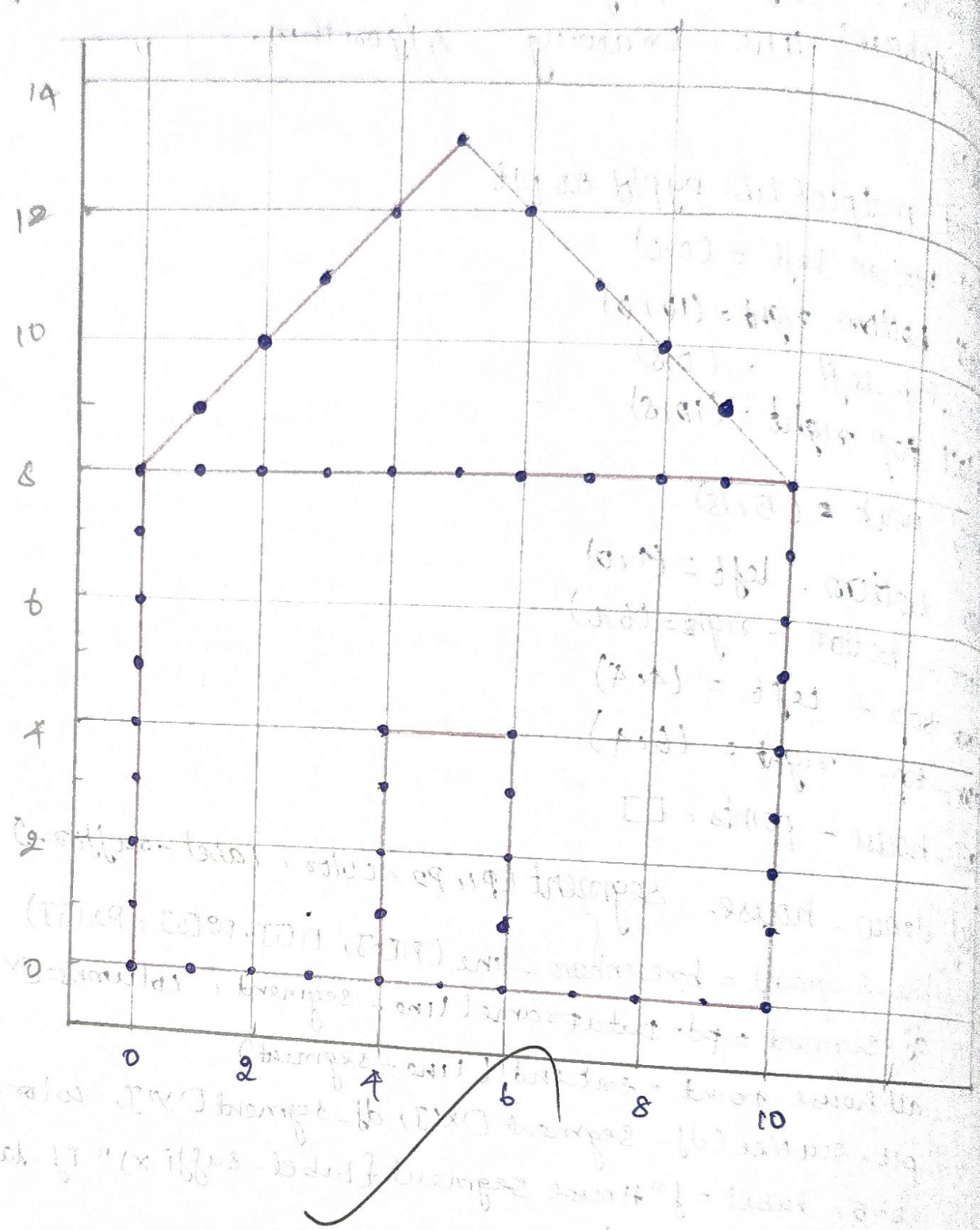
Aim:-

To write a python program to create a object using Bresenham's line Drawing Algorithm.

code:-

```
import matplotlib.pyplot as plt
wall bottom left = (0,0)
wall bottom right = (10,0)
wall top left = (0,8)
wall top right = (10,8)
roof-peak = (5,13)
door-bottom-left = (4,0)
door-bottom-right = (6,0)
door top - left = (4,4)
door-top- right = (6,4)
all_house_points = []
def draw_house_segment(p1, p2, color, label=None):
    line_segment = bresenham_line(p1[0], p1[1], p2[0], p2[1])
    df_segment = pd.DataFrame(line_segment, columns=['x','y'])
    all house point.extend(line_segment)
    plt.scatter(df_segment['x'], df_segment['y'], color=color,
    s=5, label=f"House segment {label_suffix}" if label
    seffix else None)
plt.figure(figsize=(10,10))
draw_house_segment(wall_bottom_left, wall_bottom_right, 'brown', 'Base')
draw_house_segment(wall_bottom_right, wall_top_right, 'brown', 'Rightwall')
draw_house_segment(wall_top_right, wall_top_left, 'brown', 'Top Wall')
draw_house_segment(wall_top_left, wall_Bottom_left, "brown", 'Leftwall')

draw_house_segment(wall top_left, roof_Peak, 'red', 'Roof Left')
draw_house_segment(wall_top_right, roof_Peak, 'red', 'Roof Right')
```


14
12
10
8
6
4
2
0
0
2
4
6
8
10

```
draw_house_segment(door_bottom_left, door_bottom_right, 'gray',
                   'Door Bottom')
draw_house_segment(door_bottom_right, door_top_right, 'gray',
                   'Door Right')
draw_house_segment(door_top_right, door_top_left, 'gray',
                   'Door Top')
draw_house_segment(door_top_left, door_bottom_left, 'gray',
                   'Door Left')

plt.title("2D House Drawing with Bresenham's Algorithm")
plt.xlabel("X-coordinate")
plt.ylabel("Y-coordinate")
plt.grid(True)
plt.gca().set_aspect('equal', adjustable='box')

if all_house_points:
    min_x = min(p[0] for p in all_house_points) - 1
    max_x = max(p[0] for p in all_house_points) + 1
    min_y = min(p[1] for p in all_house_points) - 1
    max_y = max(p[0] for p in all_house_points) + 1
    plt.xlim(min_x, max_x)
    plt.ylim(min_y, max_y)

plt.legend(loc='lower left', bbox_to_anchor=(1,0))
plt.tight_layout()
plt.show()
```

11/8/24

Result:-

Hence create a object was successfully implemented.

Ex-~~00~~-05 Date: 11/08/2026

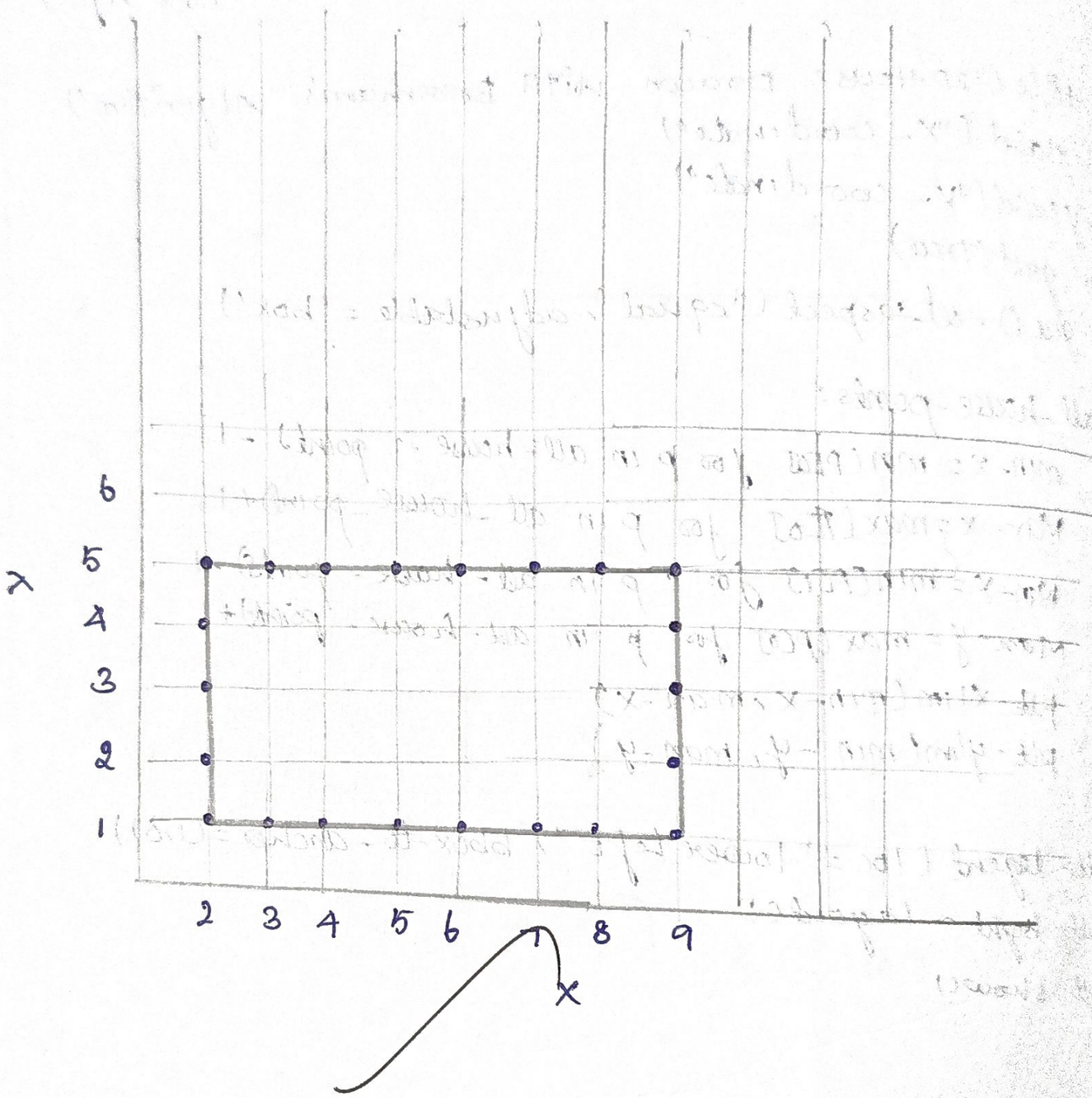
Draw a rectangle using Bresenham's Algorithm.

Aim :-

To write a python program to draw a rectangle using Bresenham's line Drawing Algorithm with the given top-left (2,2), width-8, & height 5.

program :-

```
import matplotlib.pyplot as plt
def bresenham_line(x0,y0,x1,y1):
    pt = []
    dx = abs(x1 - x0)
    dy = abs(y1 - y0)
    sx = 1 if x0 < x1 else -1
    sy = 1 if y0 < y1 else -1
    err = dx - dy
    x,y = x0,y0
    while True:
        pt.append((x,y))
        if x == x1 and y == y1:
            break
        e2 = 2*err
        if e2 > -dy:
            err = dy
            y += sy
    return pts.
def draw_rectangle(x,y, width, height):
    x0, y0 = x,y
    x1,y1 = x + width - 1, y
    x2, y2 = x + width - 1, y + height - 1
    x3,y3 = x, y + height - 1
```


y
6
5
4
3
2
1
2
3
4
5
6
8
9
x

```
top = bresenham_line ( x0, y0, x1, y1 )
right = bresenham_line (x1, y1, x2, y2)
bottom = bresenham_line (x2, y2, x3, y3)
left = bresenham_line (x3, y3, x0, y0)

return top + right[1:] + bottom[1:] + left[1:]
```

```
top_left = (2, 2)
width = 8
height = 5
plt = draw rectangle (top left [0], top_left[1], width, height)
x_points = [p[0] for p in points]
y_points = [p[1] for p in points]

plt.figure (figsize = (6, 6))
plt.plot (x_points, y_points, 'ro-')

plt.title ("Rectangle using Bresenham's Algorithm")
plt.xlabel ("x")
plt.ylabel ("y")
plt.grid (True)
plt.axis ("Equal")
plt.show
```



Result:-

Hence the rectangle was successfully drawn. using Bresenham's line drawing Algorithm with the given dimension and coordinates.

Ex-NO - 06 Date: 11/08/2020

DRAW A TRAFFIC SIGNALS USING CIRCLES

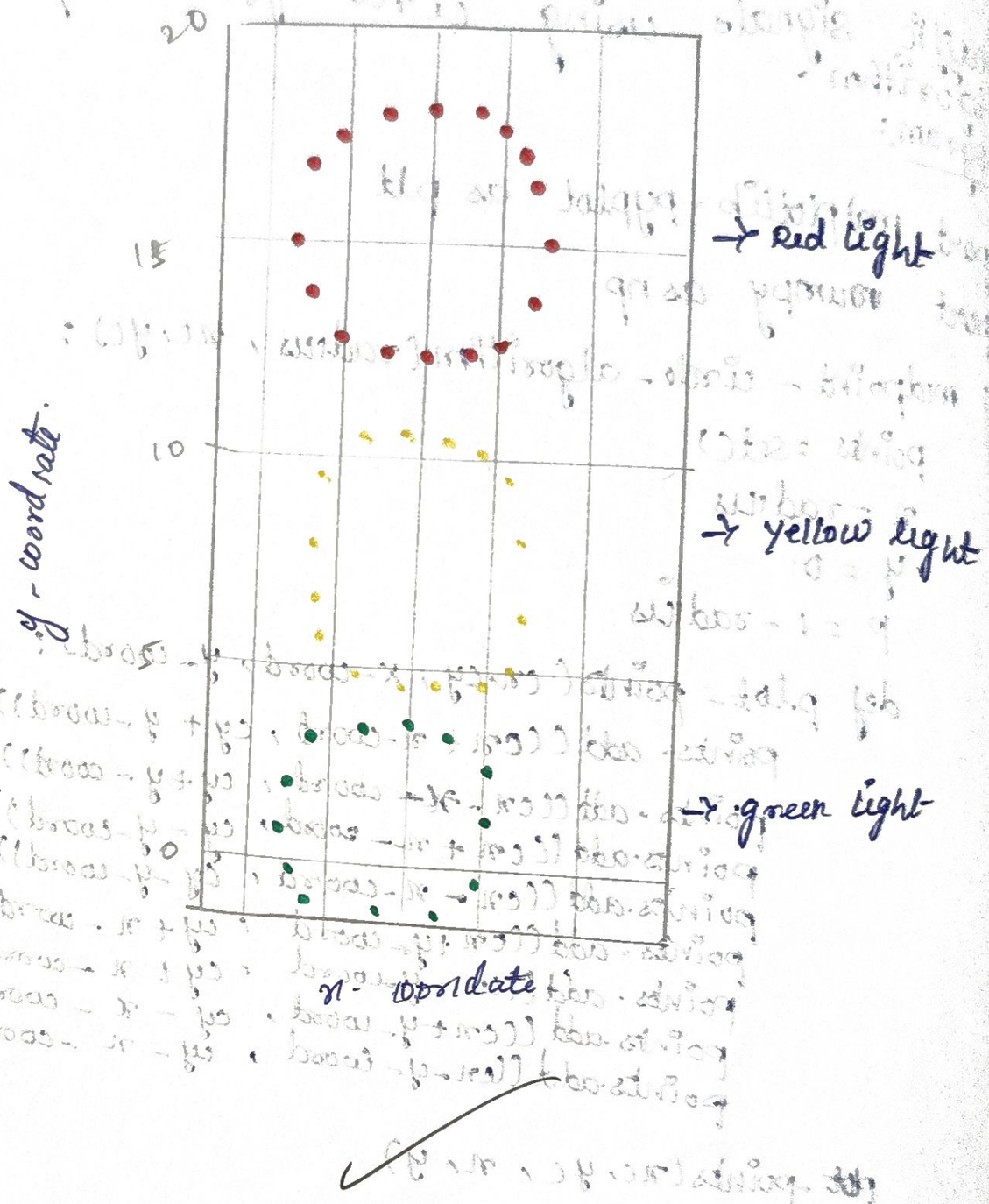
Aim:-
To write a python program to draw a traffic signals using circle for midpoint Algorithm.

program:-

```
import matplotlib.pyplot as plt
import numpy as np
def midpoint_circle_algorithm(radius, xc, yc):
    points = set()
    x = radius
    y = 0
    p = 1 - radius
    def plot_points(cx, cy, x_coord, y_coord):
        points.add((cx + x_coord, cy + y_coord))
        points.add((cx - x_coord, cy + y_coord))
        points.add((cx + x_coord, cy - y_coord))
        points.add((cx - x_coord, cy - y_coord))
        points.add((cx + y_coord, cy + x_coords))
        points.add((cx - y_coord, cy + x_coords))
        points.add((cx + y_coord, cy - x_coords))
        points.add((cx - y_coord, cy - x_coords))
    plot_points(xc, yc, x, y)
    while x > y:
        y += 1
        if p <= 0:
            p = p + 2*y + 1
        else:
            x -= 1
            p = p + 2*y - 2*x + 1
        plot_points(xc, yc, x, y)
    return sorted(list(points))

plt.figure(figsize=(4, 12))
radius = 3
center_x = 0
```

Traffic signal using Midpoint circle Algorithm



```
red_center_y = 15
red_points = midpoint_circle_algorithm(radius, center_x, red_center_y)
red_x = [p[0] for p in red_points]
red_y = [p[1] for p in red_points]
plt.scatter(red_x, red_y, color='red', s=50, label='Red Light')

Yellow_center_y = 7.5
yellow_points = midpoint_circle_algorithm(radius, center_x, yellow_center_y)
yellow_x = [p[0] for p in yellow_points]
yellow_y = [p[1] for p in yellow_points]
plt.scatter(yellow_x, yellow_y, color='yellow', s=50, label='Yellow Light')

green_center_y = 0
green_points = midpoint_circle_algorithm(radius, center_x, green_center_y)
green_x = [p[0] for p in green_points]
green_y = [p[1] for p in green_points]
plt.scatter(green_x, green_y, color='green', s=50, label='Green light')

plt.title('Traffic Signal using Midpoint circle Algorithm')
plt.xlabel('x-coordinate')
plt.ylabel('y-coordinate')
plt.grid(True)
plt.gca().set_aspect('equal', adjustable='box')

min_x_lim = center_x - radius - 2
max_x_lim = center_x + radius + 2
min_y_lim = green_center_y - radius - 2
max_y_lim = red_center_y + radius + 2
plt.xlim(min_x_lim, max_x_lim)
plt.ylim(min_y_lim, max_y_lim)
plt.legend()
plt.show()
```

Result -

Hence, successfully verified the given traffic signals using circles.

Ex No. 07 Date:- 11/8/16

Draw a smiling face using circle, face, eye, nose mouth. gui

Aim:-

To write a python code to draw a smiling face using circle, face, eye, nose mouth.

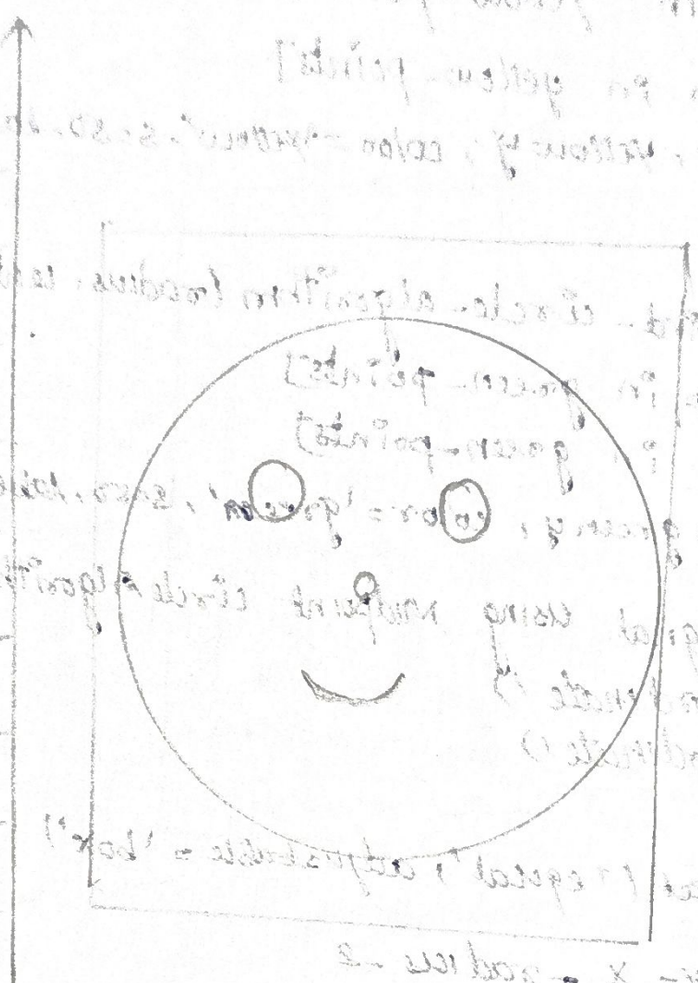
Code:-

```
import matplotlib.pyplot as plt
fig, ax = plt.subplot (figsize = (4,4))
ax.add_patch(plt.circle((0,0), 1.0, color = 'sandle', ec, black, lw
ax.add_patch(plt.circle((-0.35, 0.35), 0.12, color = 'black'))
ax.add_patch(plt.circle((0.35, 0.35), 0.12, color = 'black'))
ax.add_path(plt.circle (-0.35, 0.35, 0.12, color = 'black'))

simle-x = [0.4, -0.2, 0, 0.2, 0.4]
simle-y = [-0.2, -0.4, -0.45, -0.4, -0.2]
ax.plot(simle-x, simle-y, color = 'black', lw=3, solid-capstyle = rou
ax.set-xlim (-1.2, 1.2)
ax.set-ylim (-1.2, 1.2)
ax.axis('off')
plt.show()
```

Result:-

Here create a simling face using circle is successfully verified


y
x

EX-ON-08 Date - 11/8

Draw a olympic logo using ellipse.

Aim:-

To write a python programming to draw a olympic log using ellipse

Code:-

```
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
fig, ax = plt.subplots(figsize=(7,4))
rings = [(-2.4, 0.5), 'blue'),
         ((0.0, 0.5), 'black'),
         ((2.4, 0.5), 'red'),
         ((-1.2, -0.5), 'Yellow')
         ((1.2, -0.5), 'green')]

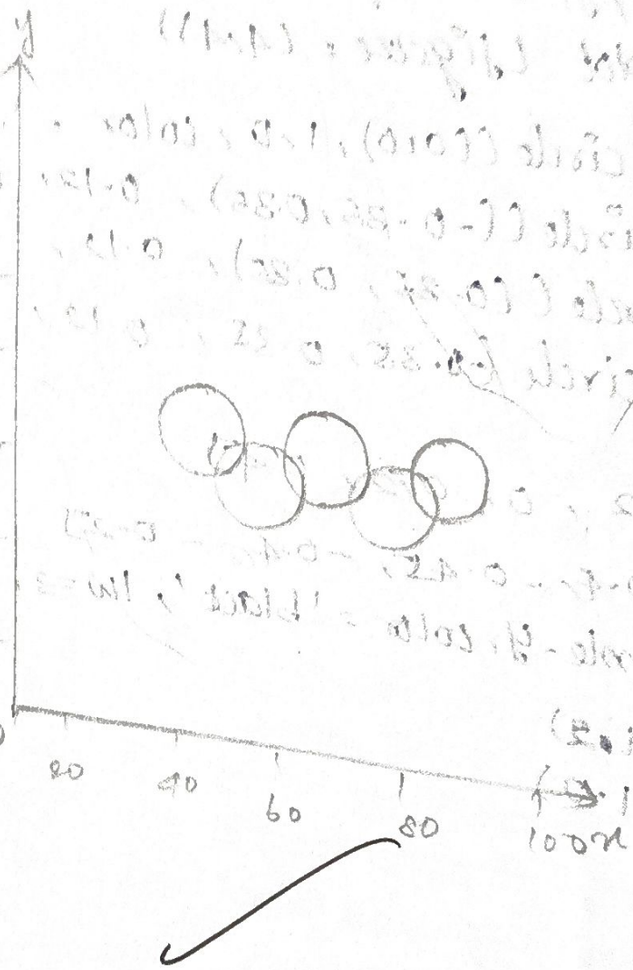
for position, color in rings:
    ax.add_patch(Ellipse(position, 2.0, 2.0, color=color, fill=
        False, lw=6))

ax.set_xlim(-4,4)
ax.set_ylim(-2,2)
ax.set_aspect('equal')
ax.axis('off')
plt.show()
```

Result:

Hence olympic logo using ellipse is verified.

output



Ex-No-09 Date: 11/8/26

Draw a butterfly using Multiple ellispes.

Aim:-

To write a python programming to draw a butterfly using Multiple ellipes

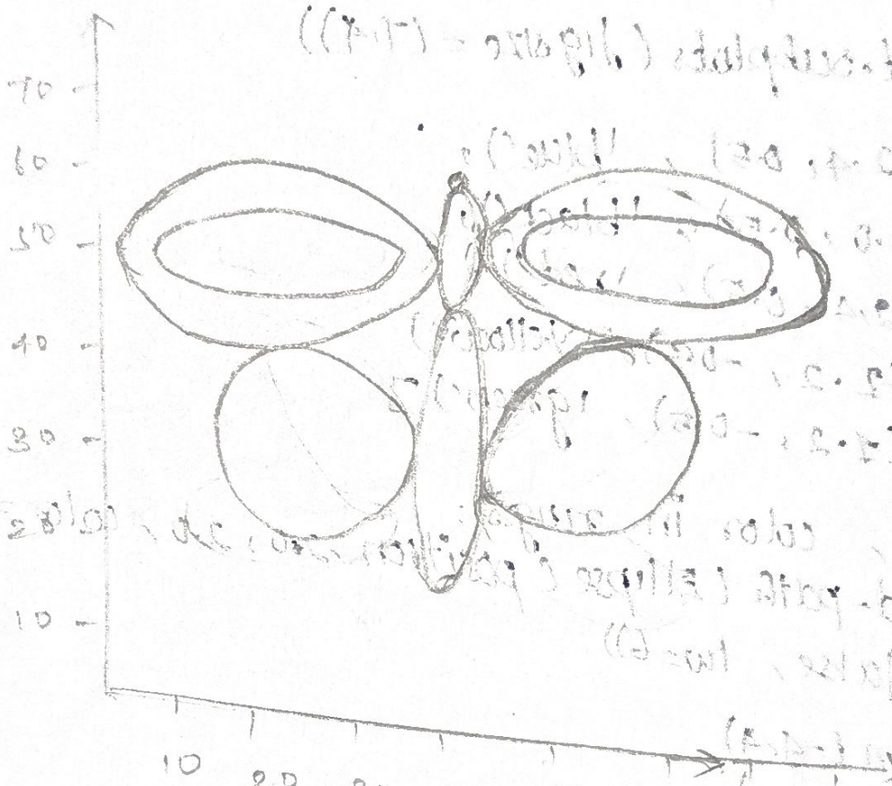
Code

```
import matplotlib.pylot as plt
from matplotlib.patches import Ellipse
fig, ax = plt.subplots(figsize=(6,6))
ax.add_patch(Ellipse((-0.6,-0.4), 1.2, 1.6, angle=40, color='deepskyblue', alpha=0.8))
ax.add_patch(Ellipse((0.6,-0.4), 1.2, 1.6, angle=-40, color='deepskyblue' alpha=0.8))
ax.add_patch(Ellipse((-0.9,0.5), 1.8, 2.2, angle=35, color='darkorange', alpha=0.9))
ax.add_patch(Ellipse((0.9,0.5), 1.8, 2.2, angle=-35, color='darkorange', alpha=0.9))
ax.add_patch(Ellipse((0.9,0.5), 1.2, 1.5, angle=35, color='gold'))
ax.add_patch(Ellipse((0.9,0.5), 1.2, 1.5, angle=-35, color='gold'))
ax.set_xlim(-2.5,2.5)
ax.set_ylim(-2.0, 2.0)
ax.set_aspect('equal')
ax.axis('off')
plt.show()
```

Result:-

Hence butterfly using multiple ellipse

Verified


70
60
50
40
30
20
10
10
20
30
40
50
60
70
80

Ex-NO-10

Date :- 8/9/25

Scan conversion of polygon and using flood fill Algorithm.

Aim:-

To implement scan conversion of polygon & use flood fill algorithm.

program:

```
Import matplotlib.pyplot as plt

P = [(2,2), (5,8), (9,3), (7,1), (3,1)]

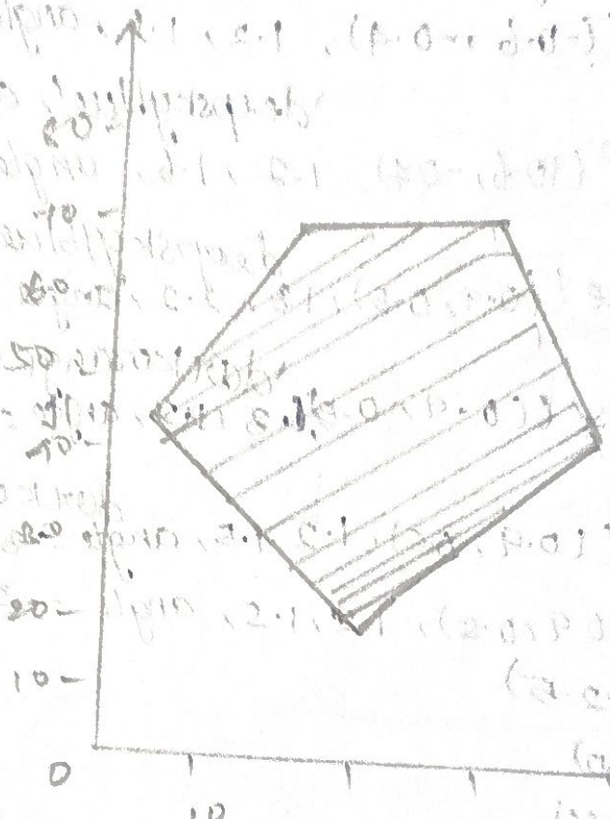
def scan_fill(p):

    ys = range(min(y for x, y in p), max(y for x, y in p)+1)

    for y in ys:
        x = []
        for i in range(len(p)):
            x1, y1 = p[i]
            x2, y2 = p[(i+1)% len(p)]
            if y1 == y2 and min(y1, y2) <= y <= max(y1, y2)
                x.append(x1 + (y-y1)*(x2-x1)/(y2-y1))
        x.sort()
        for i in range(0, len(x), 2):
            plt.plot(x[i:i+2], [y, y], 'b.')
x, y = zip(*(p + [p[0]]))
plt.plot(x, y, 'k')
scan_fill(p)
plt.fill(x, y, color = 'red', alpha = 0.1)
plt.axis('equal')
plt.title("scan conversion and flood fill")
plt.show()
```

Result:-

Hence, successfully implemented scan conversion of polygon using flood fill Algorithm.


output
0
10

Ex-NO-11 Date 11/09/25

Basic 2D transformation using 2D shapes

Aim:-

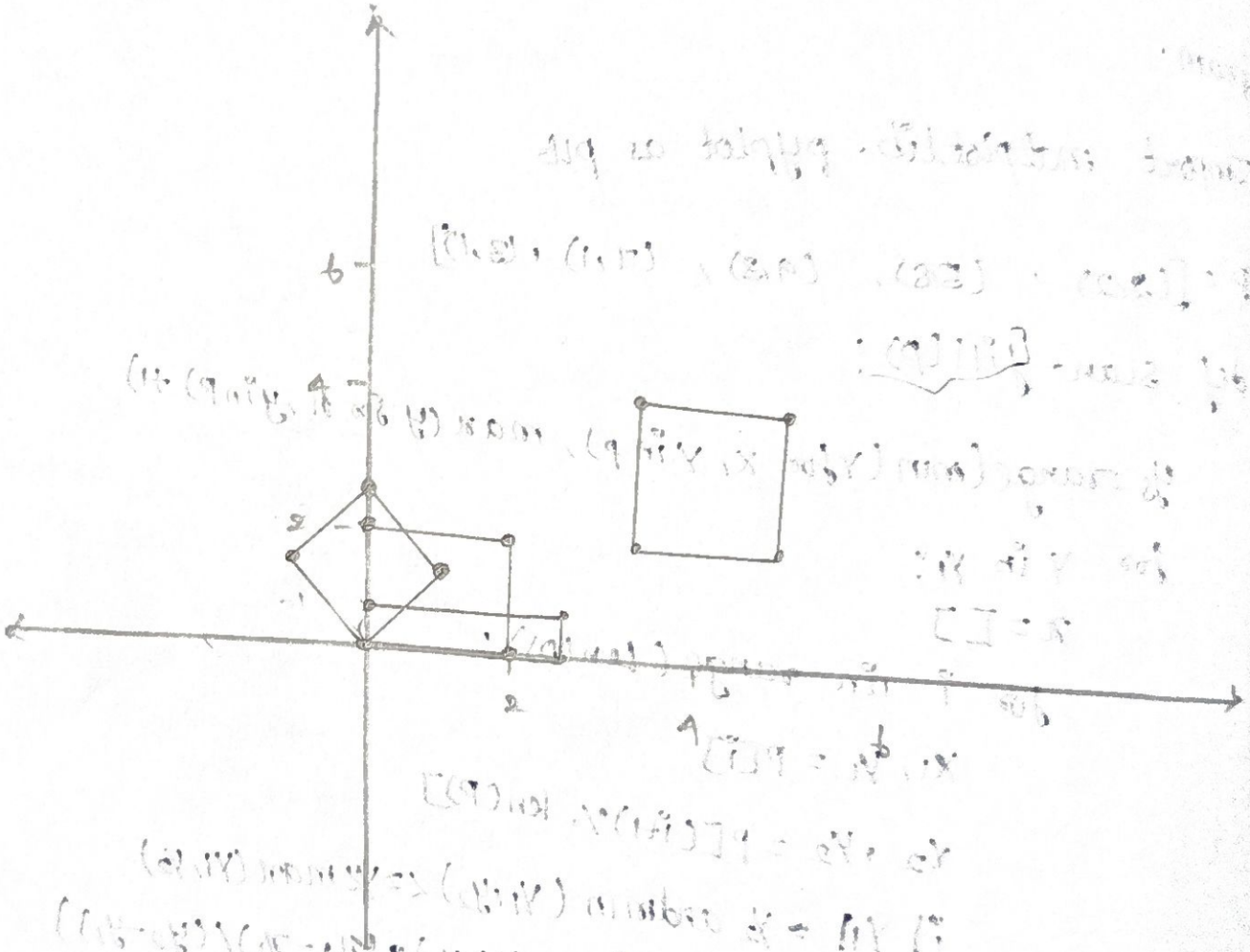
To implement basic 2D Transformation using 2D shapes

program:-

```
import matplotlib.pyplot as plt
import numpy as np
square_shape = np.array([[0,2,2,0],[0,0,2,2],[1,1,1,1]])
def plot_polygon(ax, pts, color, label):
    x = np.append(pts[0], pts[0][0])
    y = np.append(pts[1], pts[1][0])
    ax.plot(x,y, color=color, label=label, marker="o")
    ax.fill(x,y, color=color, alpha=0.2)
def translate(pts, dx, dy):
    T = np.array([[1,0,dx],[0,1,dy],[0,0,1]])
    return np.dot(T, pts)
def scale(pts, sx, sy):
    S = np.array([[sx,0,0],[0,sy,0],[0,0,1]])
    return np.dot(S, pts)
def rotate(pts, angle=deg):
    rad = np.radians(angle_deg)
    c,s = np.cos(rad), np.sin(rad)
    R = np.array([[c,-s,0],[s,c,0],[0,0,1]])
    return np.dot(R, pts)

t_square = translate(square_shape, 3,2)
s_square = scale(square_shape, 1.5, 0.5)
r_square = rotate(square_shape, 45)
```

BASIC 2D TRANSFORMATIONS ON A SQUARE



```
fig, ax = plt.subplots(figsize=(7,7))
plot_polygon(ax, square_shape, "blue", "original")
plot_polygon(ax, t_square, "green", "translated")
plot_polygon(ax, s_square, "orange", "scaled")
plot_polygon(ax, r_square, "purple", "Rotated (45°)")

ax.set_xlim(-5,8)
ax.set_ylim(-5,8)
ax.axhline(0, color="grey", linewidth=0.8)
ax.axvline(0, color="grey", linewidth=0.8)
ax.grid(True, linestyle="--", alpha=0.5)
ax.legend()
plt.title("Basic 2D tranformation on a square")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.show()
```

Result :-

Hence, successfully implemeted the given basic 2D transformation using 2D shapes.

Basic 2D transformation using Triangle.

Aim:-
To implement basic 2D transformation using Triangle.

Code:

```
import matplotlib.pyplot as plt
import numpy as np
triangle = np.array([[0,4,2],[0,0,3],[1,1,1]])
def plot_triangle(ax, pts, color, label):
    x = np.append(pts[0], pts[0][0])
    y = np.append(pts[1], pts[1][0])
    ax.plot(x,y, color=color, label=label, marker="o")
    ax.fill(x,y, color=color, alpha=0.2)
def Translate(pts, dx, dy):
    T = np.array([[1,0,dx],[0,1,dy],[0,0,1]])
    return np.dot(T, pts)
def scale(pts, sx,sy):
    s = np.array([[sx,0,0],[0,sy,0],[0,0,1]])
    return np.dot(s, pts)
def rotate(pts, angle_deg):
    rad = np.radians(angle_deg)
    c,s = np.cos(rad), np.sin(rad)
    R = np.array([[c,-s,0],[s,c,0],[0,0,1]])
    return np.dot(R, pts)

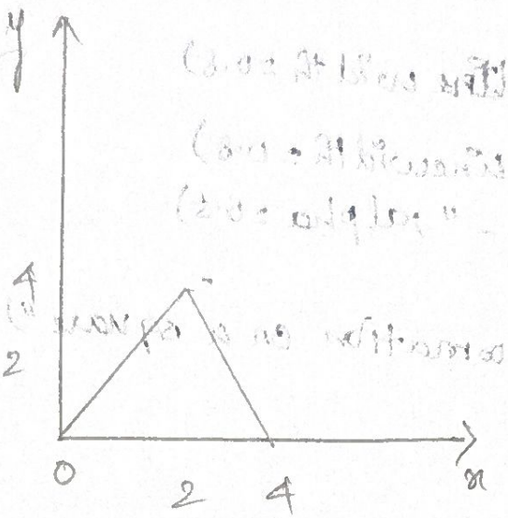
fig, ax = plt.subplots(figsize=(7,7))
plot_triangle(ax, triangle, "blue", "original")
plot_triangle(ax, t_triangle, "green", "Translated")
plot_triangle(ax, s_triangle, "orange", "Scaled")
plot_triangle(ax, r_triangle, "purple", "Rotated (45°)")

ax.set_xlim(-5,8)
ax.set_ylim(-5,8)
ax.axhline(0, color="grey", linewidth=0.8)
ax.axvline(0, color="grey", linewidth=0.8)
ax.grid(True, linestyle="--", alpha=0.5)
ax.legend()
plt.show()
```

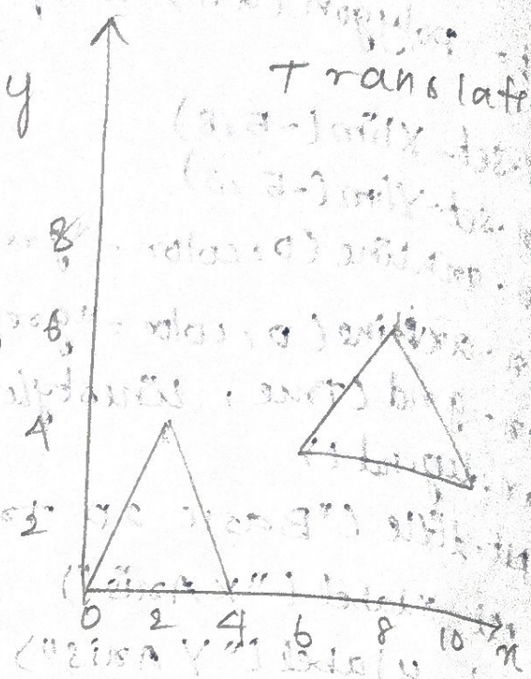
Result:-

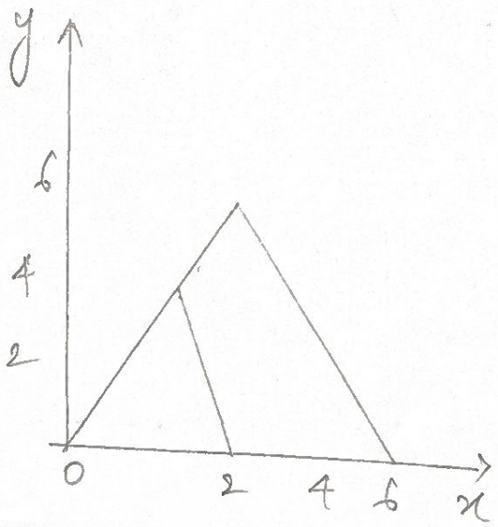
Hence successfully implemented 2D transformation using Triangle

original

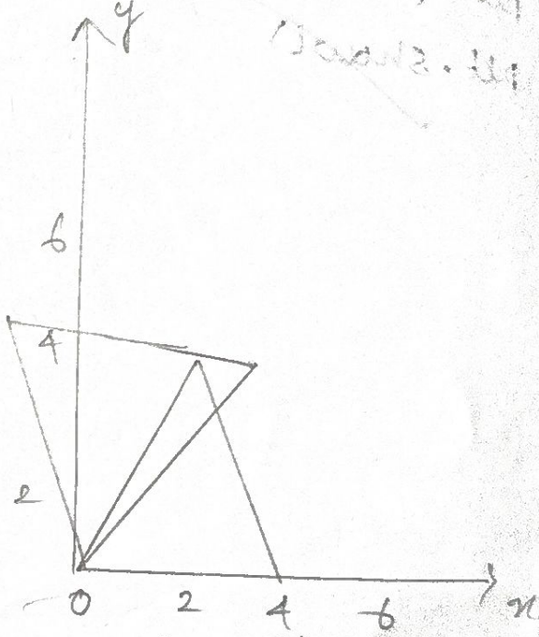


Translation

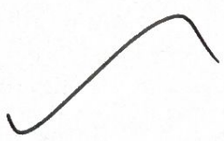




Scaling



Rotation



EX-NO.12
Date 22/9/26

POLYGON CLIPPING

Aim:-
To implement polygon clipping using python programming

PROGRAM.

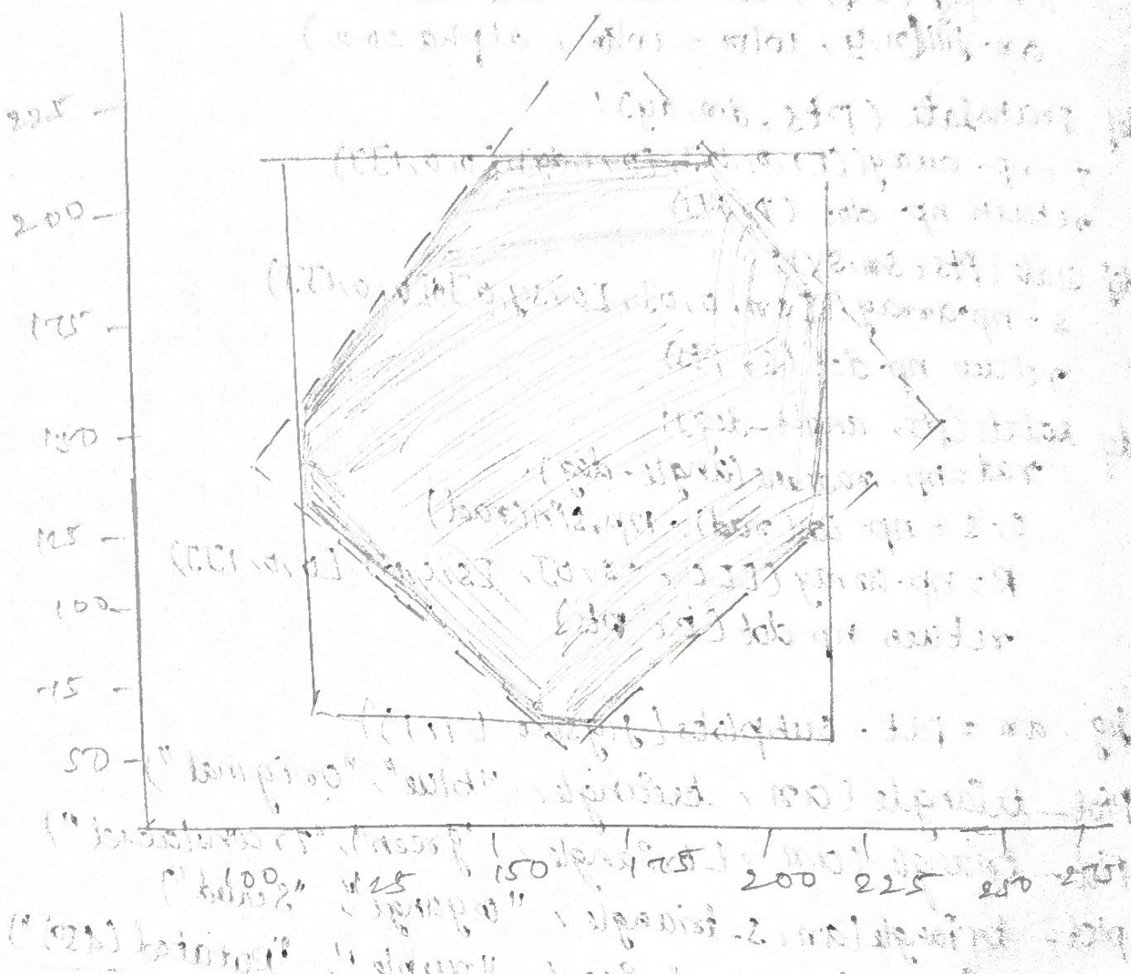
```
! pip install -q shapely matplotlib
import matplotlib.pyplot as plt
from shapely.geometry import polygon
poly_subj = Polygon([(100,150), (200,250), (300,150), (200,50)])
poly_clip = polygon([(120,80), (120,255), (280,200), (280,80)])
poly_result = poly_subj.intersection(Poly_clip)
fig, ax = plt.subplots(figsize=(6,6))
ax.plot(*poly_subj.exterior.xy, 'b--', label='original polygon')
ax.plot(*poly_clip.exterior.xy, 'g:', label='clipping window')
ax.fill(*poly_result.exterior.xy, 'r', alpha=0.5, label=
                                        'clipped result')
ax.set_title('Sutherland-Hodgman clipping')
ax.legend()
ax.grid(True)
plt.show().
```

Result:-
Hence successfully implemented polygon clipping

--- original polygon

——— clipping window

——— clipped result



Sutherland Hodgman clipping

EX:NO-13

3D Transformation using 3D shapes.

AIM:

To implement 3D transformation using 3D shapes by python programming

program:

```
import numpy as np
import matplotlib.pyplot as plt
cube = np.array([[-1,-1,-1],[1,-1,-1],[-1,1,-1],[1,1,-1],
                 [-1,-1,1],[1,-1,1],[-1,1,1],[1,1,1]])

edges = [(0,1),(0,2),(0,4),(1,3),(1,5),(2,3),(2,6),(3,7),
         (4,5),(4,6),(5,7),(6,7)]

def draw_cube(ax, points, color, name):
    for a,b in edges:
        ax.plot([points[a,0],points[b,0]],
                [points[a,1],points[b,1]],
                [points[a,2],points[b,2]],
                color=color)
    ax.scatter(points[:,0],points[:,1],points[:,2],color=color,
               label=name)

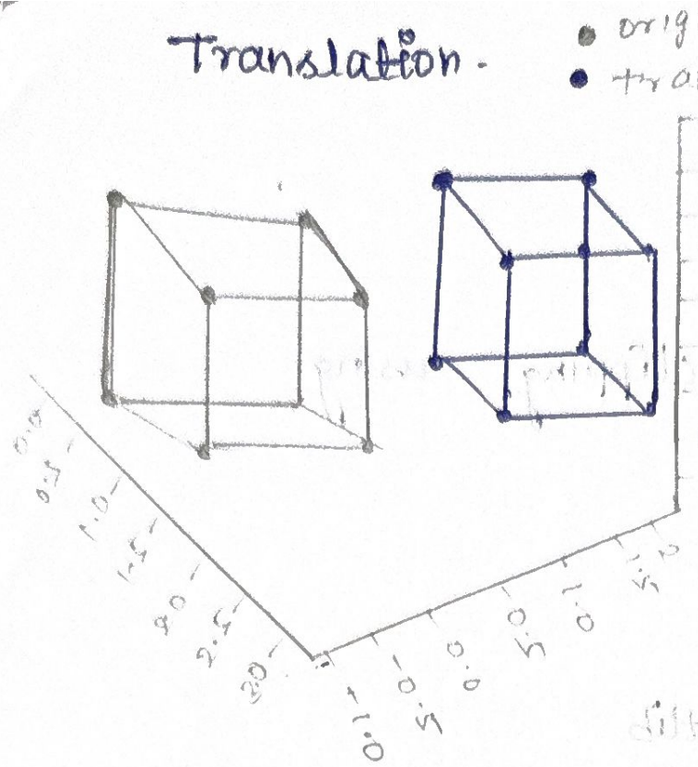
translated = cube + [2,1,0]
scaled = cube * [2,0.5,1.5]
angle = np.pi/4
R = np.array([[np.cos(angle),-np.sin(angle),0],
              [np.sin(angle),np.cos(angle),0],
              [0,0,1]])
rotated = cube @ R[45°]
shear = np.array([[1,0.5,0.3],[0,1,0],[0,0,1]])
sheared = cube @ shear

plt.tight_layout()
plt.show()
```

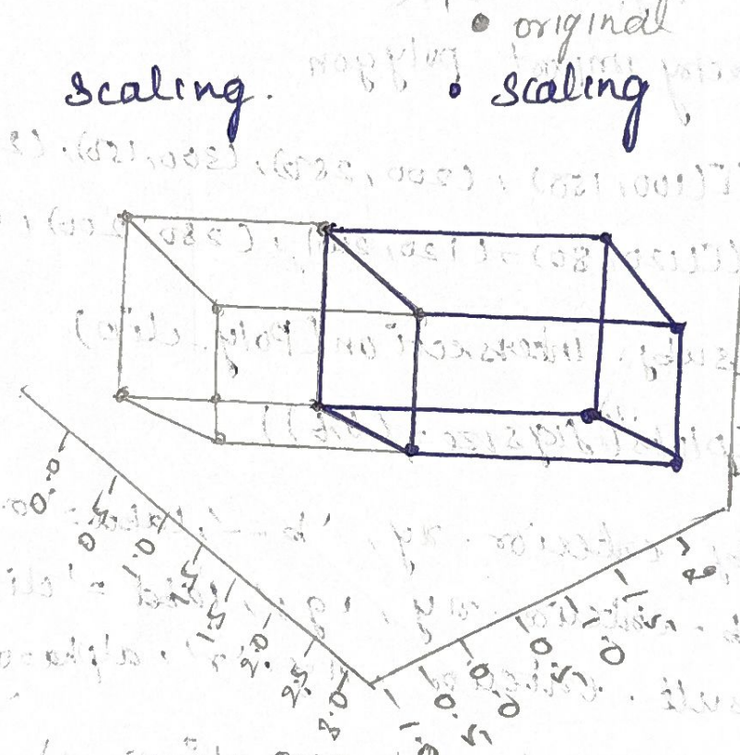
Result:-

Hence successfully implement the given 3D Transformation using 3D shapes by Python programming.

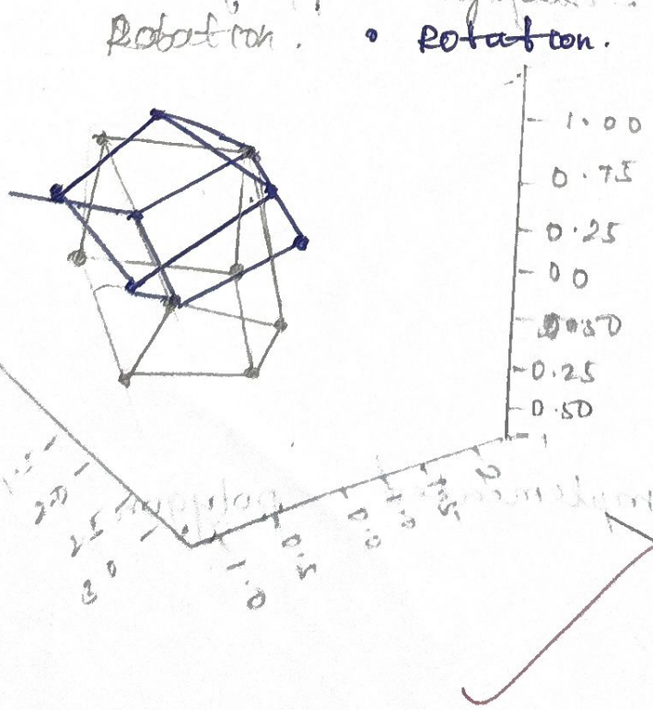
Translation.



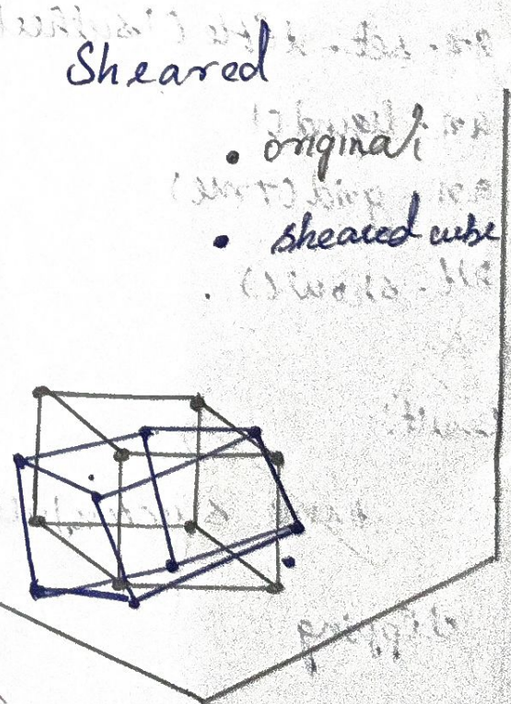
Scaling.



Rotation.



Sheared



Ex - 14 Date :- 06/10/2026

Create and Render 3D objects Using Polygons and Apply Basic color and shading Techniques

Aim:-

To create and Render 3D objects such as cube using polygonal Modeling and apply flat shading, Basic color techniques

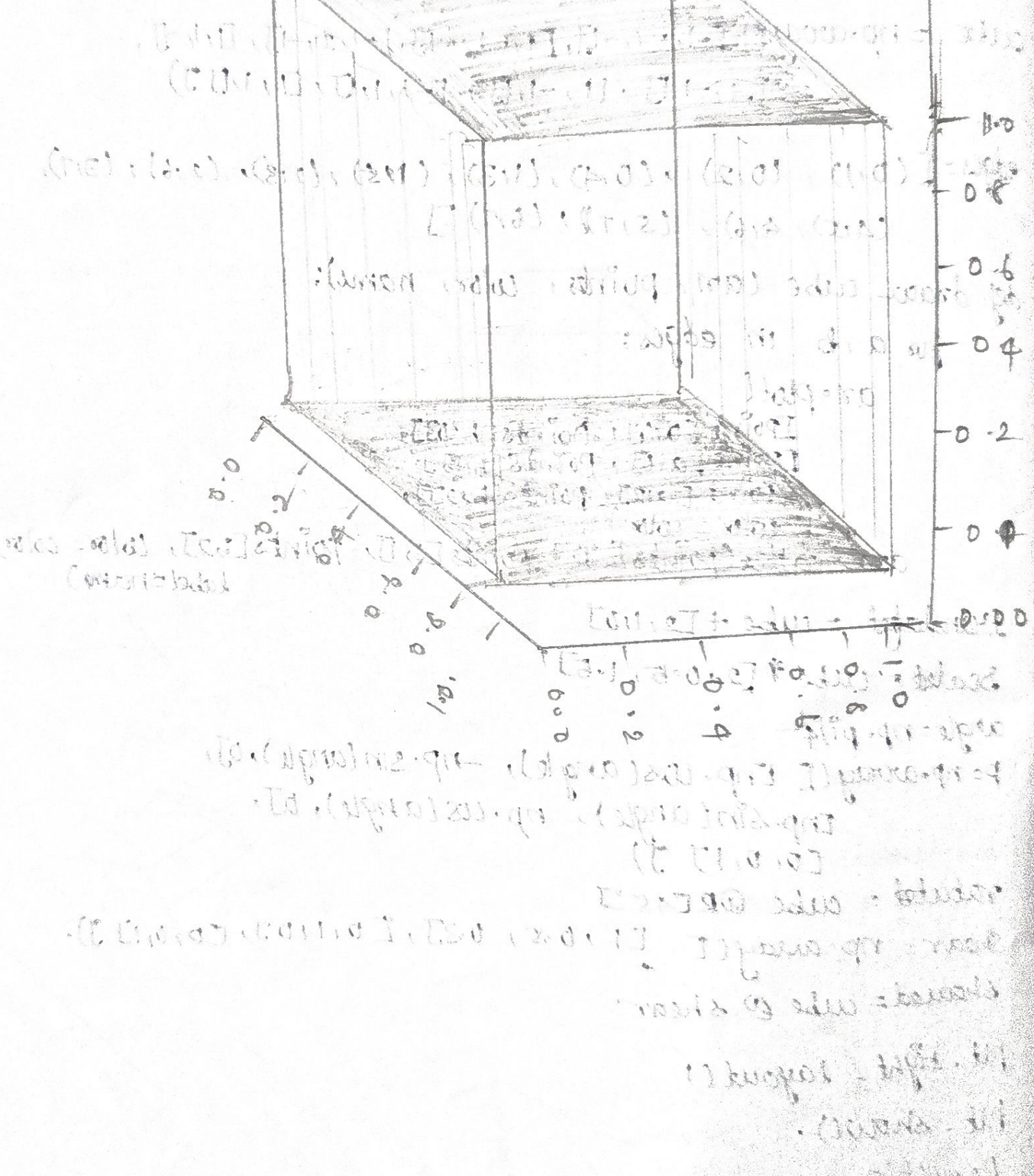
program:-

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import numpy as np.
Vertices = np.array([ [0,0,0], [2,0,0], [2,2,0], [0,2,0] [0,0,2], [2,0,2],
                      [2,2,2], [0,2,2] ])
faces = [ [vertices[i] for i in [0,1,2,3]],
          [vertices[i] for i in [4,5,6,7]],
          [vertices[i] for i in [0,1,5,4]],
          [vertices[i] for i in [2,3,7,6]],
          [vertices[i] for i in [1,2,6,5]],
          [vertices[i] for i in [4,7,3,0]] ]
colors = ['red', 'green', 'blue', 'yellow', 'orange', 'purple']
fig = plt.figure(figsize = (8,6))
ax = fig.add_subplot(111, projection = '3d')
cube = Poly3DCollection( faces, facecolors = colors, edgecolors = 'black',
       linewidths = 1 , alpha = 0.7)
ax.add_collection3d(cube)
ax.set_xlabel('X Axis')
ax.set_ylabel('Y Axis')
ax.set_zlabel('Z Axis')
ax.set_xlim(0,3)
ax.set_ylim(0,3)
ax.set_zlim(0,3)
ax.set_title('3D cube with flat shading')
plt.show()
```

Result:-

Hence successfully created 3D polygonal object & rendered.

3D cube with flat shading -



Exp-NO:-15

Multimedia Application that Integrates Images sounds and video in a simple UI.

Aim:-

To develop a simple multimedia application that integrates into a unified user interface using python and suitable libraries.

program:-

```
import tkinter as tk
from tkinter import filedialog
from PIL import Images, ImageTk
import cv2
import threading
def play_audio():
    playsound('sample_audio.mp3')
def play_video():
    cap = cv2.VideoCapture('sample_video.mp4')
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow("video", frame)
        if cv2.waitkey(25) & 0x FF == ord('q'):
            break
        cap.release()
        cv2.destoryAllWindows()
def load_image():
    img = Image.Open('sample_image.jpg')
    img = img.resize((200, 300))
    img_tk = ImageTk.PhotoImage(img)
    Panel.configure(image = img_tk)
    panel.image = img_tk
    btn_img.pack(pack=5)
    btn_audio.pack(pady=5)
    btn_video.pack(pady=5)
    Window.mainloop()
```

Result:-

A Multimedia application was successfully created.

Ex. No:- 16:-

Capture video audio from Webcam or Microphone and Display on Multimedia Interface

Aim:-

To develop a program that captures live video from the Webcam, live audio from the Microphone and display the using a simple multimedia interface.

Program:-

```
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import threading
import tkinter as tk

def record_audio():
    duration = 5
    fs = 44100
    print("Recording Audio...")
    audio = sd.rec(int(duration * fs), samplerate = fs, channels=2)
    sd.wait()
    write("recorded_audio.wav", fs, audio)
    print("Audio recording saved.")

def capture_video():
    cap = cv2.VideoCapture(0)
    print("Press 'q' to stop video.")
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        cv2.imshow('Webcam Feed', frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cap.release()
    cv2.destroyAllWindows()
    window.tk.
window = tk.Tk()
window.title("Capture Audio and Video")
btn_audio.pack(pady=10)
btn_video.pack(pady=10)
window.mainloop()
```

Result:-

Live video and audio were successfully captured.

Exp No: 11

Create an Application to send and Receive Multimedia Messages (Text, Image, Audio).

Aim:-

To develop a basic chat-like application sending and receiving text, images, and audio.

program:-

```
import tkinter as tk
from tkinter import filedialog, scrolledtext
from PIL import Image, ImageTk
from playsound import playsound
import threading
def send_text():
    msg = text_entry.get()
    if msg:
      chat_box.insert(tk.END, f"You: {msg}\n")
      text_entry.delete(0, tk.END)
def send_image():
    path = filedialog.askopenfilename(filetypes=[("Image Files", "*.jpg *.png *.jpeg")])
    if path:
        image_label.pack()
        chat_box.image_create(tk.END)
def send_audio():
        path = filedialog.askopenfilename(filetypes=[("Audio Files", "*.mp3 *.wav")])
        if path:
          chat_box.insert
          threading.Thread(target=lambda: playsound(path)).start()
window = tk.Tk()
window.title("Multimedia Messaging App")
window.geometry("400x500")
chatbox.pack()
chat_frame.pack(pady=10)
text_entry = tk.Entry(window, width=30)
text_entry.pack(side=tk.LEFT, padx=5)
btn_text.pack(side=tk.LEFT)
btn_image.pack(side=tk.LEFT)
btn_audio.pack(side=tk.LEFT)
window.mainloop()
```

Result:

A multimedia messaging interface was successfully created.

EX-NO-18

3D Model using Blender and apply Basic shading and Textures.

Aim:-
To create a simple 3D model (eg. a house or object) using Blender, and apply Basic shading, Textures.

Procedure:-

1. Launch Blender.
2. Model creation.
   - Use Add > mesh to insert basic shapes like cube, cylinder, plane, etc.
   - Combine these object to form a house or a similar structure.
   - use Edit Mode for extruding and shaping
3. Shading.
   - switch to the shading workspace.
   - Add a new material for each object.
   - Adjust Base color, Specular and Roughness.
4. Texturing Mapping
   - In the shader Editor, add an Image Texture node.
   - Load an external image.
   - connect it to the Base color
5. Lighting and camera:
   - Add light source
   - position the camera to frame the model.
6. Render Setting
   - choose cycle or Excee rendering engine
   - Adjusting resolution and sampling
   - Click Render > Render Image to produce the output.

Result:-

3D object was successfully modeled in Blender.